



Islington college
(इस्लिङ्टन कलेज)

L3NLP01_AI Data Structure and Algorithms_SU_25

Student Name: Viom Lal Shrestha

Project Title: Text-Based Adventure Game (Dungeon Crawler)

London Met ID: 23048612

College ID: NP01AI4A230041

Assignment Due Date: Thursday, August 28, 2025

Assignment Submission Date: Wednesday, August 27, 2025

Submitted to: Mr. Mahotsav Bhattarai

GitHub Link	https://github.com/Viom-Shrestha/Dungeon_Crawler
--------------------	---

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

23048612 Viom Lal Shrestha DSA Summer.docx

Islington College,Nepal

Document Details

Submission ID

trn:oid:::3618:109886973

Submission Date

Aug 28, 2025, 9:47 AM GMT+5:45

Download Date

Aug 28, 2025, 9:48 AM GMT+5:45

File Name

23048612 Viom Lal Shrestha DSA Summer.docx

File Size

15.2 KB

7 Pages

2,345 Words

12,942 Characters



Page 1 of 10 - Cover Page

Submission ID trn:oid:::3618:109886973



Page 2 of 10 - Integrity Overview

Submission ID trn:oid:::3618:109886973

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- 5 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
Matches that are still very similar to source material
- 0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 1% Internet sources
- 0% Publications
- 1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 5 Not Cited or Quoted 2%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1% Internet sources
- 0% Publications
- 1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	kopia.io	<1%
2	Submitted works	The University of Wolverhampton on 2022-04-29	<1%
3	Submitted works	University of Bedfordshire on 2024-04-25	<1%

Table of Contents

1. Introduction: Dungeon Crawler	1
2. Aims and Objectives	2
3. Expected Outcome and Deliverance	2
4. Project Risks, Threats, and Contingency Plans	3
5. Methodology	4
6. Resource Requirements	4
7. Tools and Technology Used	4
8. Dashboard	5
8.1 launcher.py	5
8.2 dungeon_crawler.py	6
8.3 dundeon_crawler_gui.py	17
9. Conclusion	28

Table of Figures

Figure 1: Launcher.py main interface	5
Figure 2: Exiting Launcher	5
Figure 3: CLI difficulty selection	6
Figure 4: CLI Easy mode setup	7
Figure 5: CLI Normal Map	8
Figure 6: CLI Hard Map	9
Figure 7: player movement in CLI.....	10
Figure 8: CLI fog of war	11
Figure 9: CLI Inventory interface	11
Figure 10: Equipping items in CLI	12
Figure 11: CLI Inventory after equipping an item.....	12
Figure 12: CLI help command.....	13
Figure 13: CLI quests command.....	13
Figure 14: CLI status command.....	13
Figure 15: Treasure and Quest Completion in CLI	14
Figure 16: CLI combat log.....	14
Figure 17: CLI Player level up status	14
Figure 18: Reaching next floor in CLI.....	15
Figure 19: Validation condition of game boundary in CLI	16
Figure 20: Undo feature in CLI	16
Figure 21: GUI loading screen.....	17
Figure 22: GUI main menu	18
Figure 23: GUI easy mode map	18
Figure 24: GUI normal mode map.....	19
Figure 25: GUI hard mode map	19
Figure 26: GUI with multiplayer off.....	20
Figure 27: fog of war in GUI	20
Figure 28: GUI boundary control	21
Figure 29: Obtaining treasure in GUI	21
Figure 30: Quest completion mechanism in GUI	22

Figure 31: Quests in GUI.....	22
Figure 32: GUI inventory.....	23
Figure 33: GUI healing mechanism.....	23
Figure 34: Equipping items in GUI	24
Figure 35: Selling items	24
Figure 36: GUI undo button in action.....	25
Figure 37: When a player dies in multiplayer.....	25
Figure 38: Death Screen.....	26
Figure 39" Final dungeon level.....	26
Figure 40: Victory screen.....	27
Figure 41: High score.....	27

Table of Tables

Table 1: Risks, Threats and Contingency Plan 3

1. Introduction: Dungeon Crawler

This Text Based Adventure Game-Dungeon Crawler is a Python-based text and graphical adventure game where players explore dungeons, fight monsters, collect treasures, and complete quests. This project focuses on developing a text-based adventure game, also known as a dungeon crawler and showcase it using a GUI element with the help of Pygame. The main objective of this project is to build a functional game, while also implementing it to reinforce fundamental data structures and algorithms taught in the DSA Summer Industry Enrichment Classes which includes:

- Lists to store player items and simulate inventory.
- Matrix which is a 2D list to make the game map.
- Recursion to implement the fog-of-war map exploration system.
- Functions and loops for the main game and each functionality in the game.
- Linked lists to store a quest log and recent event log.
- Stacks for turn based combat system.
- Queues for implementing the undo feature.

To make the project accessible, two user interfaces were implemented:

1. Command-Line Interface (CLI): A text-driven dungeon crawler game played on the command line.
2. Graphical User Interface (GUI): A GUI based dungeon crawler game that relies on some classes of CLI game while using Pygame for visual representation and additional features.

The project integrates game design principles with algorithmic structures to provide a functioning prototype of a dungeon crawler which is a sub-genre of role-playing games (RPGs). The implementation was created to satisfy both technical and academic requirements. Additional game features includes multiplayer toggle, background music, sound effects, game mode difficulty selection for map size and monster power scaling, equipping weapons and armour on the players to increase hp and attack, use of items to restore hp, separate control settings for multiplayer, random generation of enemies and loot, quest tracking, entity sprites for visual enhancement, player status tracker, boss battle, and level wise player power scaling along with hp restore upon level up.

2. Aims and Objectives

Aim: The aim of this project is to implement a classic dungeon crawler adventure game using Python and DSA concepts to deliver a working game that works on Command Line Interface and Graphical User Interface

Objectives: The objectives of this project are:

- i. Creating game map using a 2D Matrix and Player Inventory using 1D List.
- ii. Creating a fog of war map reveal system using recursion.
- iii. Creating a quest log and action log using linked list/
- iv. Making a “Movement Undo” feature using stack
- v. Managing combat using Queue.
- vi. Making a basic CLI game version and a more advanced GUI version.
- vii. Adding Sprites for entities and objects.
- viii. Adding Background music and sound effects.
- ix. Implementing a 2-player game feature in GUI.
- x. Implementing a game difficulty selection and player status tracking.
- xi. Implementing item usage functions along with player level up feature.
- xii. Using randomizer for enemy and loot generation.
- xiii. Creating a high score tracker.

3. Expected Outcome and Deliverance

The expected outcomes are:

- i. An active CLI-based Dungeon Crawler game with basic gameplay features.
- ii. A full GUI version with screens, buttons, sprites, music, and additional features.
- iii. Integration and demonstration of data structures such as lists, matrices, linked lists, stacks, and queues.
- iv. Improved learning of programming concepts through firsthand practice while developing the game.
- v. Comfortability in developing and handling both CLI and GUI interfaces on the same game engine.

The deliverances are:

- i. Launcher application to select either CLI or GUI versions.
- ii. CLI-based Dungeon Crawler game.
- iii. GUI-based Dungeon Crawler game.
- iv. Project report with documentation, and figures.

4. Project Risks, Threats, and Contingency Plans

Table 1: Risks, Threats and Contingency Plan

S. No.	Risks/Threats	Contingency Plan
1.	GUI crashes	Testing each gui code, providing a CLI fallback and adding exception handling.
2.	GUI layout issues	Simplifying GUI design to match the core concepts in CLI engine.
3.	Feature Creeps	Making an initial feature list through requirement analysis and sticking to it.
4.	Map generation errors (Player stuck in wall, unreachable treasures, boss not spawning)	Adding boundary checks, spawn logic, random generation validation, and force spawning of necessary elements.
5.	Players not leveling up	Adding player experience checks after monster combat.
6.	Assets availability	Using placeholders for fallback.
7.	Game Balance	Adding monster hp, defense, and attack scaling with floor level as well as player stat scaling with level up.

5. Methodology

This Dungeon Crawler game was developed using Agile-like process. Instead of a set plan, the project was carried out in small, bite-sized parts. The data structures lists, recursion, linked lists, stacks, and queues and their implementation was the natural "sprint," where new features were introduced, tested, and integrated into the system.

This iterative process allowed flexibility. The features were developed, updated, or tweaked as needed, and they were being tested after every implementation cycle on a continuous basis. Lots of changes and improvements were handled, implemented, and discarded in the development cycles during the development process.

6. Resource Requirements

- **Hardware:** Any PC with >4GB RAM, keyboard for CLI and GUI, mouse for GUI
- **Software:**
 - Python 3.11+
 - Pygame 2.5
 - Code Editor (VS code)
 - Standard libraries (Queue, Random, Enum)
- **Assets:** PNG sprites, sound effects.

7. Tools and Technology Used

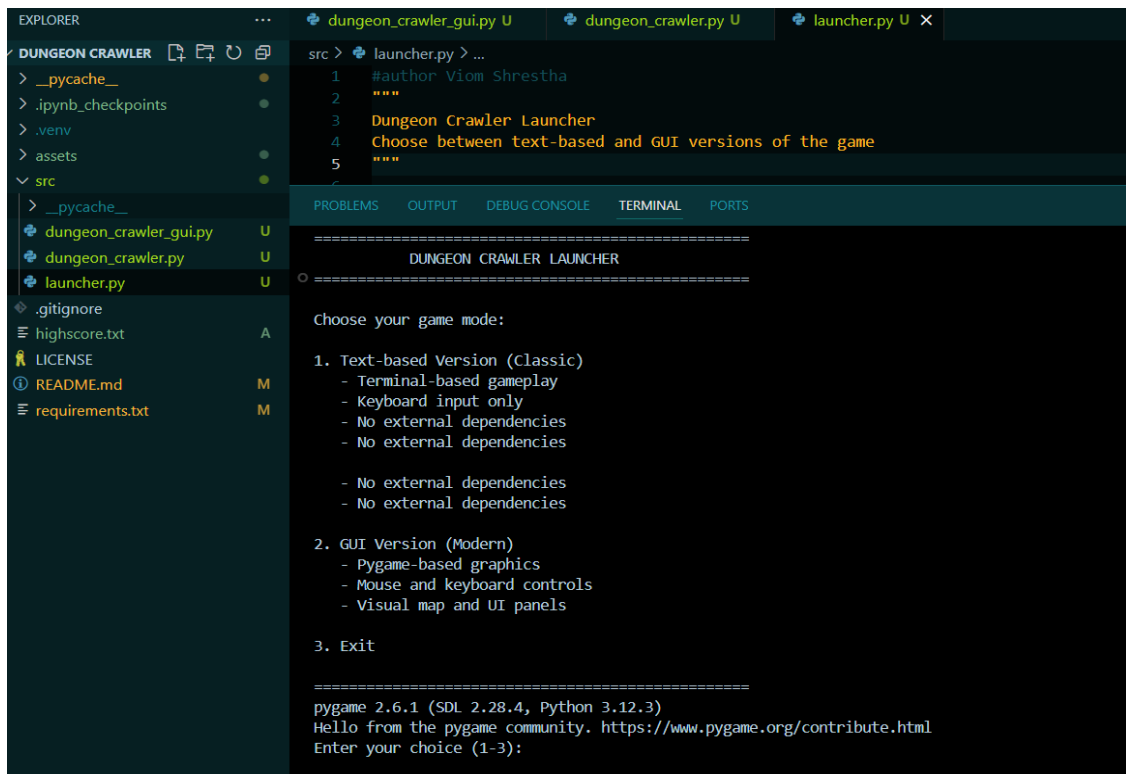
The tools and technology used are:

- **Programming Language:** Python
- **Libraries:** Pygame (for GUI), Enum, Random, Time, and other standard libraries.
- **Version Control:** GitHub
- **IDE:** VSCode

8. Dashboard

8.1 launcher.py

Launcher.py provides an easy command line feature that allows the player to choose between the CLI (Command-Line Interface) and the GUI (Graphical User Interface) versions of the game.



The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The file `launcher.py` is selected. The editor window shows the code for `launcher.py`, which includes a docstring and a main function. The terminal window at the bottom shows the output of running `launcher.py`, which displays a menu for choosing the game mode.

```
src > launcher.py > ...
1 #author Viom Shrestha
2 """
3 Dungeon Crawler Launcher
4 Choose between text-based and GUI versions of the game
5 """
6

=====
DUNGEON CRAWLER LAUNCHER
=====

Choose your game mode:

1. Text-based Version (Classic)
- Terminal-based gameplay
- Keyboard input only
- No external dependencies
- No external dependencies

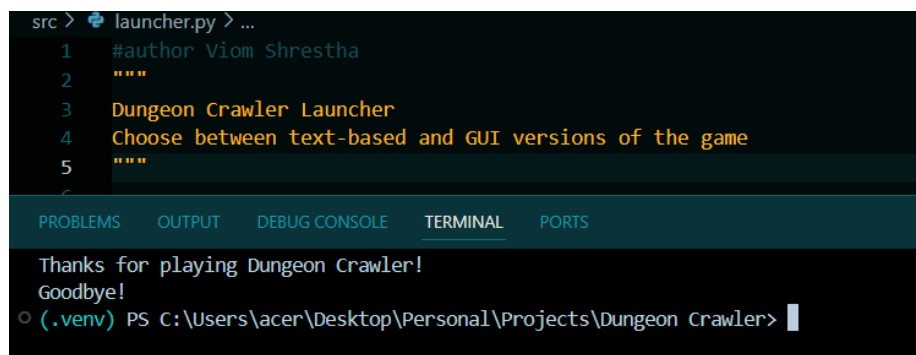
2. GUI Version (Modern)
- Pygame-based graphics
- Mouse and keyboard controls
- Visual map and UI panels

3. Exit

=====
pygame 2.6.1 (SDL 2.28.4, Python 3.12.3)
Hello from the pygame community. https://www.pygame.org/contribute.html
Enter your choice (1-3):
```

Figure 1: Launcher.py main interface

This page allows the user to select the text based version of the game by pressing 1, GUI version by pressing 2 and exit the program by pressing 3.



The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The file `launcher.py` is selected. The editor window shows the code for `launcher.py`, which includes a docstring and a main function. The terminal window at the bottom shows the output of running `launcher.py`, which displays a menu for choosing the game mode.

```
src > launcher.py > ...
1 #author Viom Shrestha
2 """
3 Dungeon Crawler Launcher
4 Choose between text-based and GUI versions of the game
5 """
6

=====
DUNGEON CRAWLER LAUNCHER
=====

Choose your game mode:

1. Text-based Version (Classic)
- Terminal-based gameplay
- Keyboard input only
- No external dependencies
- No external dependencies

2. GUI Version (Modern)
- Pygame-based graphics
- Mouse and keyboard controls
- Visual map and UI panels

3. Exit

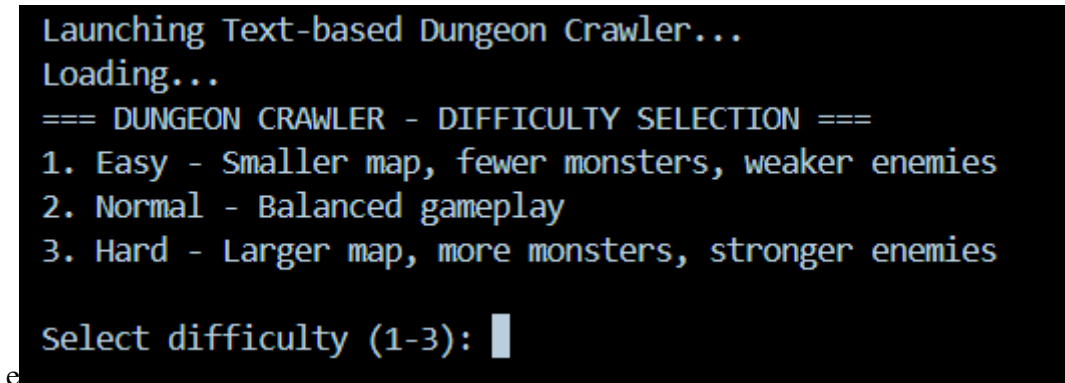
=====
pygame 2.6.1 (SDL 2.28.4, Python 3.12.3)
Hello from the pygame community. https://www.pygame.org/contribute.html
Enter your choice (1-3):
```

Figure 2: Exiting Launcher

Exiting the launcher simply clears the terminal screen, shows a goodbye message and quits the program.

8.2 dungeon_crawler.py

The `dungeon_crawler.py` is the text-based version of the game and also acts as the base for many classes and function that are used by the GUI.



```
Launching Text-based Dungeon Crawler...
Loading...
=== DUNGEON CRAWLER - DIFFICULTY SELECTION ===
1. Easy - Smaller map, fewer monsters, weaker enemies
2. Normal - Balanced gameplay
3. Hard - Larger map, more monsters, stronger enemies

Select difficulty (1-3): █
```

Figure 3: CLI difficulty selection

The first part of the program that's shown is the difficulty selection that prompts the player to choose the game difficulty mode(Easy, Normal, Hard) that determines the game map size and monster power scaling.

```

Select difficulty (1-3): 1
Welcome to Dungeon Crawler!
Difficulty: Easy
Map Size: 15x10
Type 'help' for available commands.
Good luck!

=== AVAILABLE COMMANDS ===
Movement: north/w, south/s, east/d, west/a
Actions: use <item>, inventory/i, status/s
Game: quests/q, undo/u, help/h, quit/exit

=== ITEM USAGE ===
• Use 'use <item>' to equip weapons/armor or consume potions
• Weapons increase attack, armor increases defense
• Potions restore health
None

=== LEVEL 1 ===
Legend: #=Wall, T=Treasure, M=Monster, d=Stairs, P=Player
=====
|###???????????|
|#P  ??????????|
|#   ??????????|
|# #???????????|
|# #???????????|
|?  ????????????|
|???????????????|
|???????????????|
|???????????????|
|???????????????|
|???????????????|
|???????????????|
=====

=== PLAYER STATUS ===
Health: 100/100
Level: 1
Experience: 0
Gold: 0
Attack: 15 (Base: 15)
Defense: 10 (Base: 10)
Position: (1, 1)
Difficulty: Easy

What would you like to do? █

```

Figure 4: CLI Easy mode setup

Easy mode initializes a smaller 15x10 sized map, providing a beginner friendly and explorative environment where the user can explore the game mechanics with weaker monsters.

```

Select difficulty (1-3): 2
Welcome to Dungeon Crawler!
Difficulty: Normal
Map Size: 20x15
Type 'help' for available commands.
Good luck!

=== AVAILABLE COMMANDS ===
Movement: north/w, south/s, east/d, west/a
Actions: use <item>, inventory/i, status/s
Game: quests/q, undo/u, help/h, quit/exit

=== ITEM USAGE ===
• Use 'use <item>' to equip weapons/armor or consume potions
• Weapons increase attack, armor increases defense
• Potions restore health
None

=== LEVEL 1 ===
Legend: #-Wall, T=Treasure, M=Monster, d=Stairs, P=Player
=====
|#####|
|P#     |
|# M    |
|#      |
|#      |
|d      |
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
=====

=== PLAYER STATUS ===
Health: 100/100
Level: 1
Experience: 0
Gold: 0
Attack: 15 (Base: 15)
Defense: 10 (Base: 10)
Position: (1, 1)
Difficulty: Normal

What would you like to do? █

```

Figure 5: CLI Normal Map

The normal difficulty mode creates a balanced map size of 20x15 and enemy scaling, offering a standard gameplay experience with normal monsters.

The player moves with the WASD keys.

```
What would you like to do? d
Monster action: lizard patrols
Monster action: lizard searches for prey

=== LEVEL 1 ===
Legend: #=Wall, T=Treasure, M=Monster, d=Stairs, P=Player
=====
|#####??????????|
|# P  ???????????|
|##   ???????????|
|#    ???????????|
|?    ???????????|
|?    ???????????|
|????????????????|
|????????????????|
|????????????????|
|????????????????|
|????????????????|
|????????????????|
=====

=== PLAYER STATUS ===
Health: 100/100
Level: 1
Experience: 0
Gold: 0
Attack: 15 (Base: 15)
Defense: 10 (Base: 10)
Position: (2, 1)
Difficulty: Easy

What would you like to do? █
```

Figure 7: player movement in CLI

Players move through the dungeon using WASD keys or using directions like “east”, “west”, “north”, and “south”. Each move updates the map and may trigger encounters or discoveries.

The fog of war mechanic makes it so that the map is covered with ‘?’ and gets revealed near a certain radius from the player.

```
What would you like to do? d
Monster action: jinn patrols
Monster action: jinn searches for prey

=== LEVEL 1 ===
Legend: #=Wall, T=Treasure, M=Monster, d=Stairs, P=Player
=====
|#####??????|
|#      ??????|
|##      ??????|
|#      d  ??????|
|?      ??????|
|??     ## ??????|
|??#     #  ???|
|??      P   ???|
|???     # T#  ???|
|????#####????|
=====

=== PLAYER STATUS ===
Health: 100/100
Level: 1
Experience: 0
Gold: 0
Attack: 15 (Base: 15)
Defense: 10 (Base: 10)
Position: (8, 7)
Difficulty: Easy

What would you like to do? s
```

Figure 8: CLI fog of war

The fog of war hides unexplored areas with “?” symbols. Only tiles within a set radius around the player are revealed, encouraging exploration.

```
What would you like to do? inventory

=== INVENTORY ===
1. Iron Sword - A sharp iron sword (Weapon) +5 ATK (Value: 50)
2. Steel Sword - A superior steel sword (Weapon) +8 ATK (Value: 80)

=== EQUIPPED ===
Weapon: None
Armor: None
```

Figure 9: CLI Inventory interface

Typing the “inventory” command displays the items the player has collected, including weapons, armor, and consumables.

```

What would you like to do? use iron sword
Equipped Iron Sword! Attack increased to 20

=== LEVEL 1 ===
Legend: #=Wall, T=Treasure, M=Monster, d=Stairs, P=Player
=====
|####???????????|
|## # ???????????|
|#   ???????????|
|# P  ???????????|
|#   #???????????|
|#   ????????????|
|?   ????????????|
|????????????????|
|????????????????|
|????????????????|
=====

=== PLAYER STATUS ===
Health: 100/100
Level: 1
Experience: 0
Gold: 0
Attack: 20 (Base: 15)
Defense: 10 (Base: 10)
Position: (2, 3)
Difficulty: Easy

```

Figure 10: Equipping items in CLI

The “use [item]” command equips an item from the inventory, updating the player’s stats accordingly. Using a potion heals the player hp.

```

=== INVENTORY ===
1. Steel Sword - A superior steel sword (Weapon) +8 ATK (Value: 80)

=== EQUIPPED ===
Weapon: Iron Sword (+5 ATK)
Armor: None

```

Figure 11: CLI Inventory after equipping an item

After equipping, the inventory updates to show equipped items and remaining resources along with their base values.

```
What would you like to do? help

=== AVAILABLE COMMANDS ===
Movement: north/w, south/s, east/d, west/a
Actions: use <item>, inventory/i, status/s
Game: quests/q, undo/u, help/h, quit/exit

=== ITEM USAGE ===
• Use 'use <item>' to equip weapons/armor or consume potions
• Weapons increase attack, armor increases defense
• Potions restore health
```

Figure 12: CLI help command

The “help” command lists available commands, serving as an in-game manual for new players.

```
What would you like to do? quests

=== ACTIVE QUESTS ===
1. [o] Jinn Hunter
   Defeat 1 demon
   Reward: {'gold': 50}
2. [✓] Treasure Collector
   Collect 1 treasures
   Reward: {'item': 'steel_sword'}
3. [o] Demon Slayer
   Defeat the dragon
   Reward: {'item': 'magic_shield'}
```

Figure 13: CLI quests command

The “quests” command displays current quests, progress, and completion status.

```
What would you like to do? status

=== PLAYER STATUS ===
Health: 100/100
Level: 1
Experience: 0
Gold: 0
Attack: 15 (Base: 15)
Defense: 10 (Base: 10)
Position: (1, 1)
Difficulty: Easy
```

Figure 14: CLI status command

The “status” command provides player statistics such as health, attack, defense, experience position and level.

```

What would you like to do? d
You found a treasure!
Added Iron Sword to inventory!
Added Steel Sword to inventory!
Quest 'Treasure Collector' reward: Steel Sword
Quest Completed: Treasure Collector
Monster action: snake patrols
Monster action: snake searches for prey

```

Figure 15: Treasure and Quest Completion in CLI

Discovering treasure chests and completing quests rewards players with gold, items, and progression.

```

What would you like to do? d
A jinn appears!

=== COMBAT: jinn ===
You deal 10 damage to jinn!
jinn deals 1 damage to you!
You deal 10 damage to jinn!
jinn deals 1 damage to you!
You deal 10 damage to jinn!
jinn deals 1 damage to you!
You deal 10 damage to jinn!
You defeated jinn!
Rewards: +40 EXP, +30 Gold
Quest 'Jinn Hunter' reward: +50 Gold
Quest 'Jinn Hunter' reward: +50 EXP
Quest Completed: Jinn Hunter
Combat ended. Your health: 97
LEVEL UP! You are now level 2!
Monster action: demon patrols
Monster action: demon searches for prey

```

Figure 16: CLI combat log

The combat log records turn-by-turn interactions during battle, showing attacks, damage dealt, and outcomes. Upon reaching the experience threshold and leveling up, the player's stats increase, and the log records the growth for reference.

```

=== PLAYER STATUS ===
Health: 97/110
Level: 2
Experience: 30
Gold: 80
Attack: 15 (Base: 17)
Defense: 10 (Base: 12)
Position: (7, 6)
Difficulty: Easy

```

Figure 17: CLI Player level up status

```

What would you like to do? w
You found stairs leading deeper into the dungeon!
Welcome to level 2!
Monster action: lizard patrols
Monster action: lizard searches for prey

=== LEVEL 2 ===
Legend: #=Wall, T=Treasure, M=Monster, d=Stairs, P=Player
=====
|####????????????|
|#P  M????????????|
|#  #????????????|
|#  ?????????????|
|? ?????????????|
|????????????????|
|????????????????|
|????????????????|
|????????????????|
|????????????????|
|????????????????|
=====

=== PLAYER STATUS ===
Health: 97/110
Level: 2
Experience: 30
Gold: 80
Attack: 15 (Base: 17)
Defense: 10 (Base: 12)
Position: (1, 1)
Difficulty: Easy

What would you like to do?

```

Figure 18: Reaching next floor in CLI

Using stairs allows the player to progress to deeper dungeon levels with tougher challenges.

```

What would you like to do? a
You can't walk through walls!

=== LEVEL 2 ===
Legend: #=Wall, T=Treasure, M=Monster, d=Stairs, P=Player
=====
|#####|
|#P M|
|# #|
|# ||
|? ||
|?|
|?|
|?|
|?|
|?|
|?|
|?|
|?|
|?|
|?|
=====

```

Figure 19: Validation condition of game boundary in CLI

Boundary checks prevent the player from walking through walls or invalid tiles.

```

=== LEVEL 2 ===
Legend: #=Wall, T=Treasure, M=Monster, d=Stairs, P=Player
=====
|#####|
|# PM ||
|# #|
|# ||
|? ||
|?|
|?|
|?|
|?|
|?|
|?|
|?|
|?|
|?|
|?|
=====

=== PLAYER STATUS ===
Health: 97/110
Level: 2
Experience: 30
Gold: 80
Attack: 15 (Base: 17)
Defense: 10 (Base: 12)
Position: (3, 1)
Difficulty: Easy

What would you like to do? undo
Undid move. Back at position (2, 1)

=== LEVEL 2 ===
Legend: #=Wall, T=Treasure, M=Monster, d=Stairs, P=Player
=====
|#####|
|# P M ||
|# #|
|# ||
|? ||
|?|
|?|
|?|
|?|
|?|
|?|
|?|
|?|
|?|
|?|
=====

```

Figure 20: Undo feature in CLI

The undo command reverts the player's last move using stack-based backtracking.

8.3 dundeon_crawler_gui.py

The `dundeon_crawler_gui.py` is the graphical wrapper built on top of the same game engine, but has its own new features, functions, and implementations as well as logic. It uses buttons for movement and actions, and a display area shows the dungeon, messages, quests, player status and hp bar, recent activities, and inventory. The GUI provides a more user-friendly experience with sprites and sounds while maintaining the same mechanics as the CLI version as well as an additional feature of multiplayer.

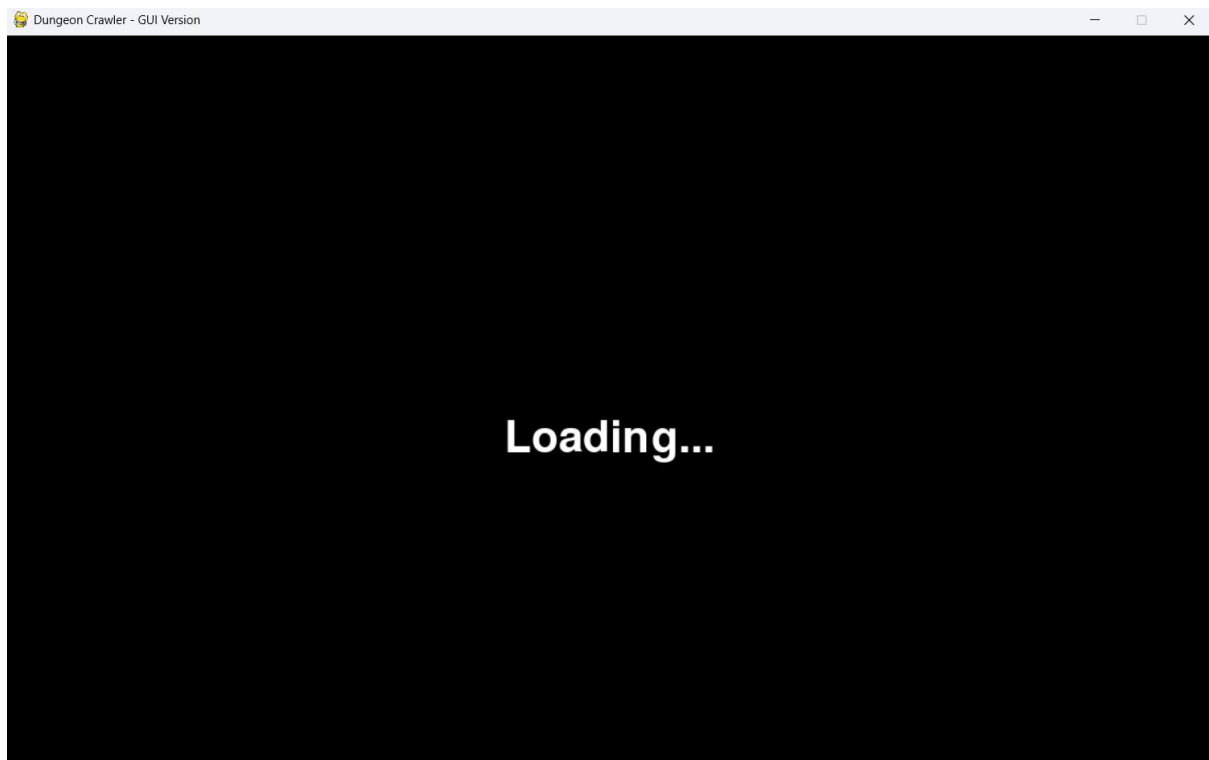


Figure 21: GUI loading screen

The GUI version begins with a loading screen, preparing assets and presenting a polished start.

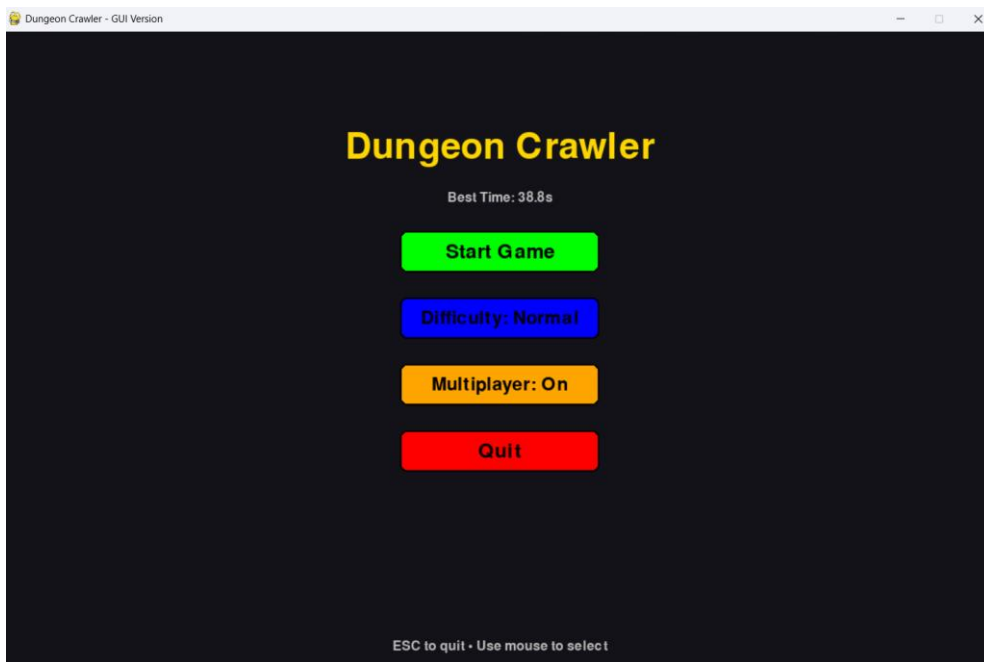


Figure 22: GUI main menu

The main menu provides options to start the game, select difficulty, toggle multiplayer, or exit.

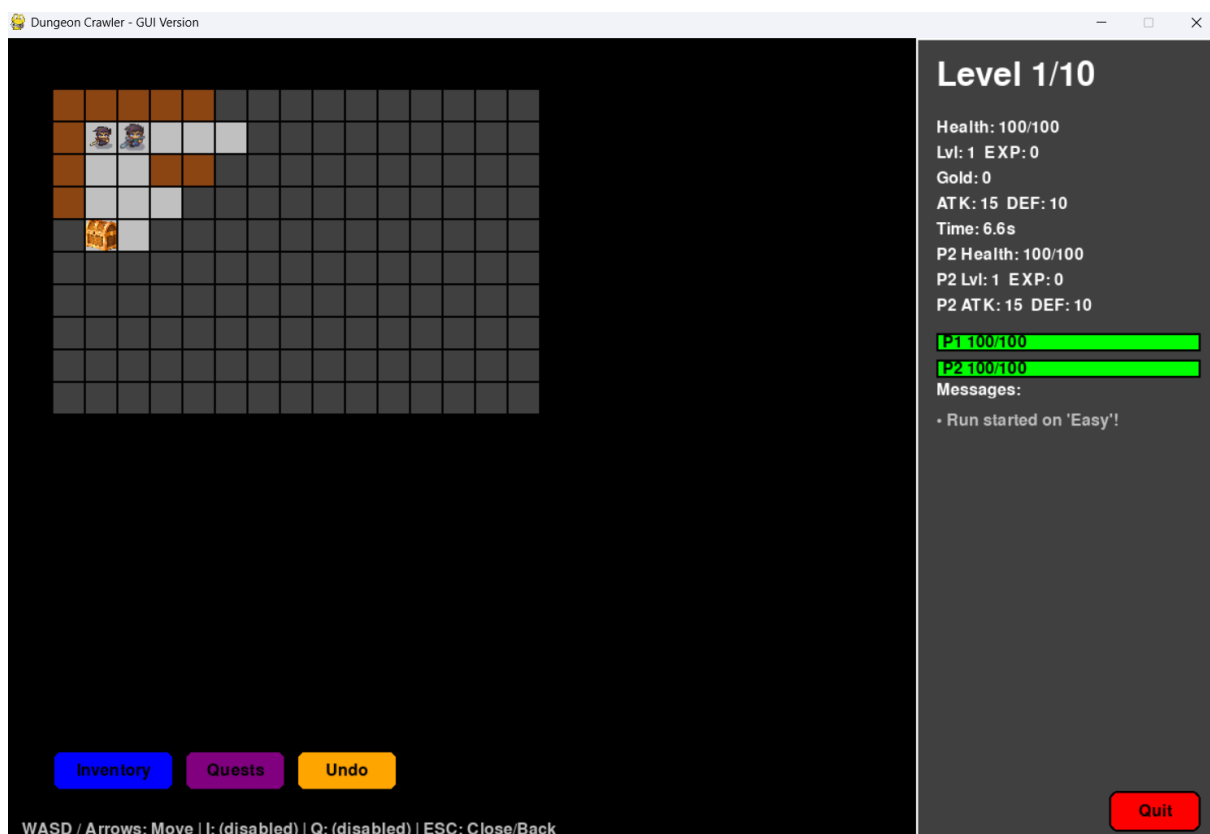


Figure 23: GUI easy mode map

The easy mode map in GUI offers a smaller dungeon with weaker monsters for beginners with a less challenging dungeon environment.

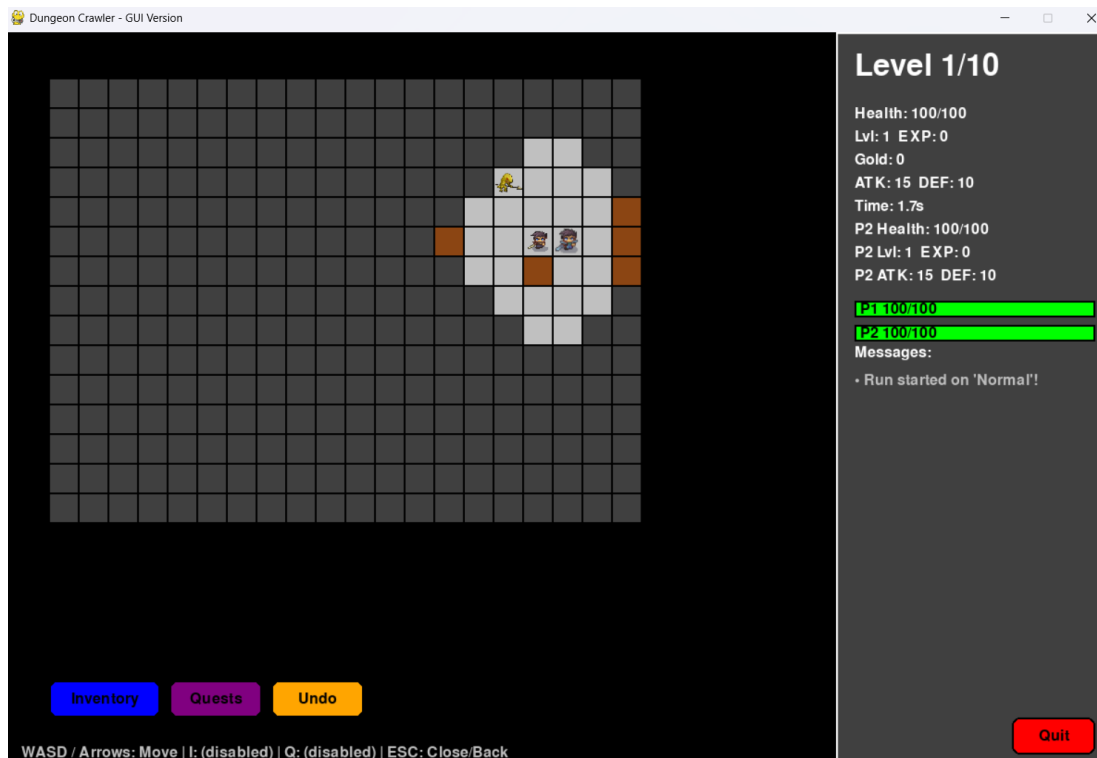


Figure 24: GUI normal mode map

Normal mode creates a balanced map layout with moderate difficulty.

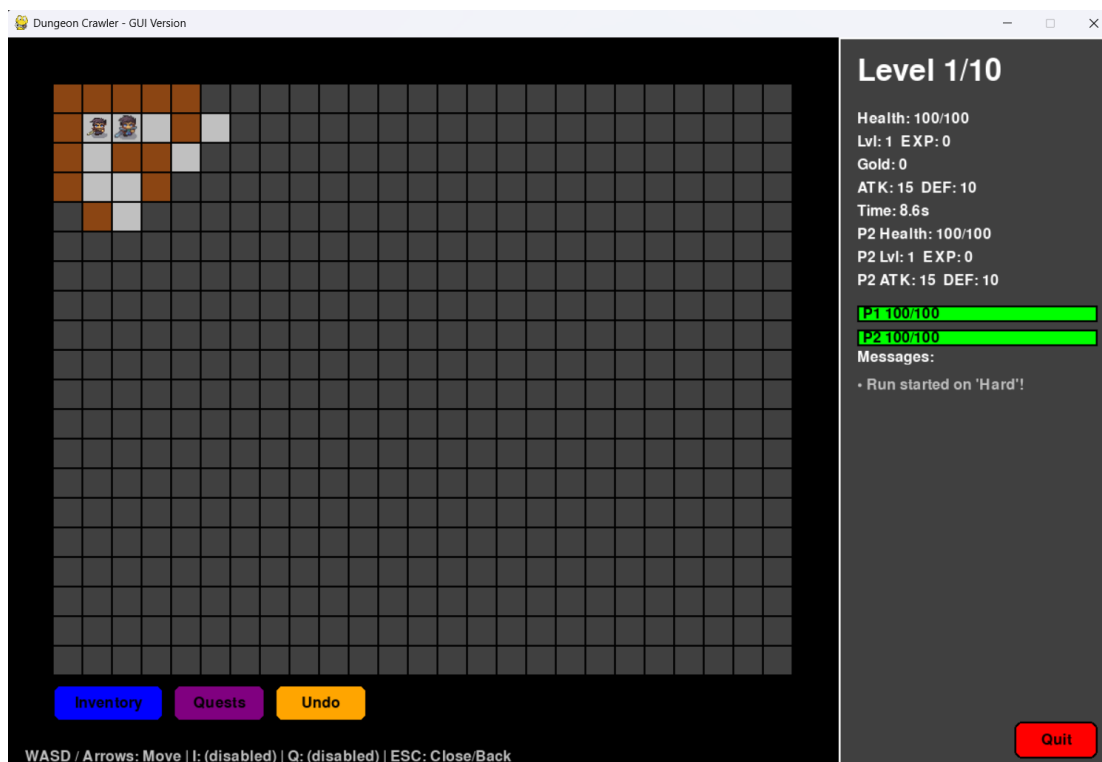


Figure 25: GUI hard mode map

Hard mode generates a large and challenging dungeon with stronger monsters.

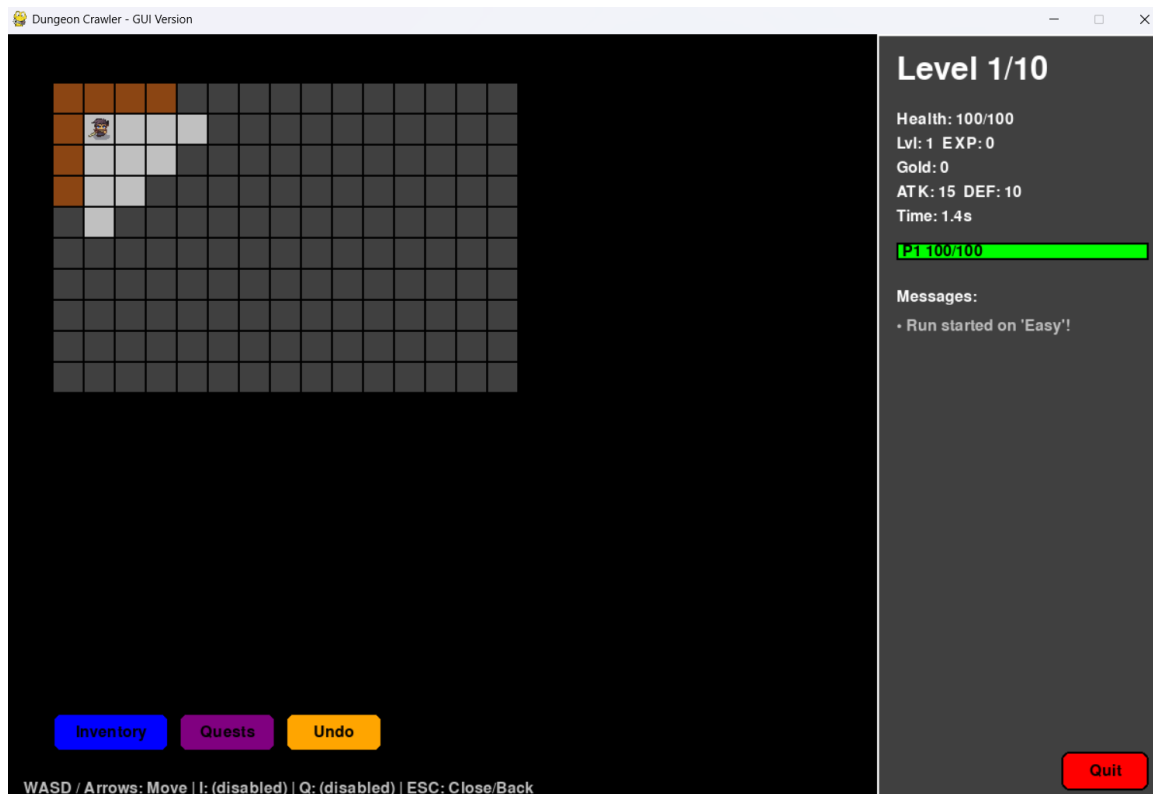


Figure 26: GUI with multiplayer off

The game can be played solo with multiplayer features disabled.



Figure 27: fog of war in GUI

Unexplored areas are hidden, with nearby tiles gradually revealed as the player moves in a certain radius.

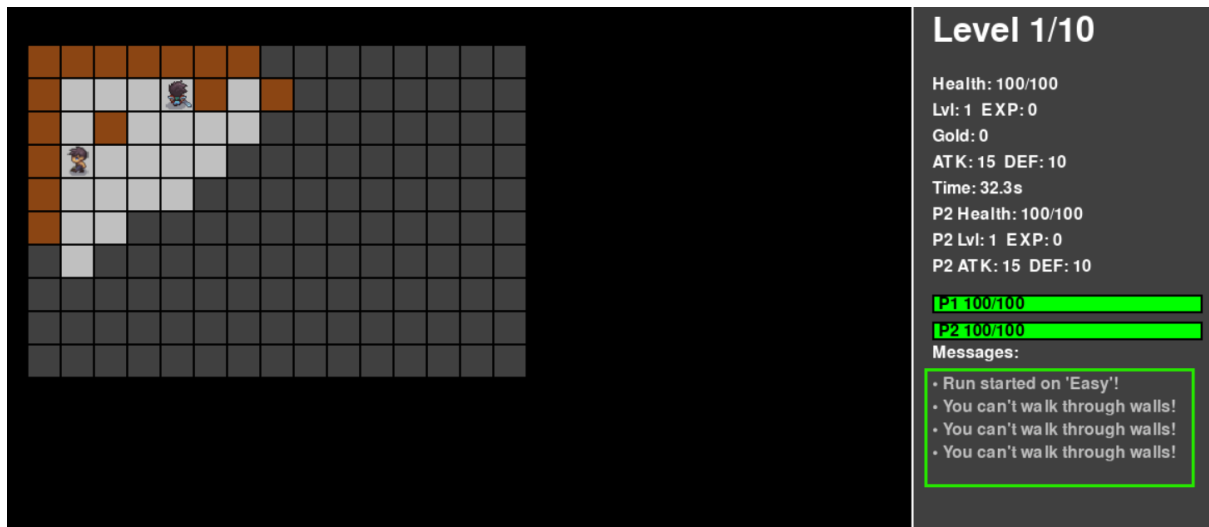


Figure 28: GUI boundary control

Boundary validation prevents the player from moving outside the playable area with a GUI warning on the screen.

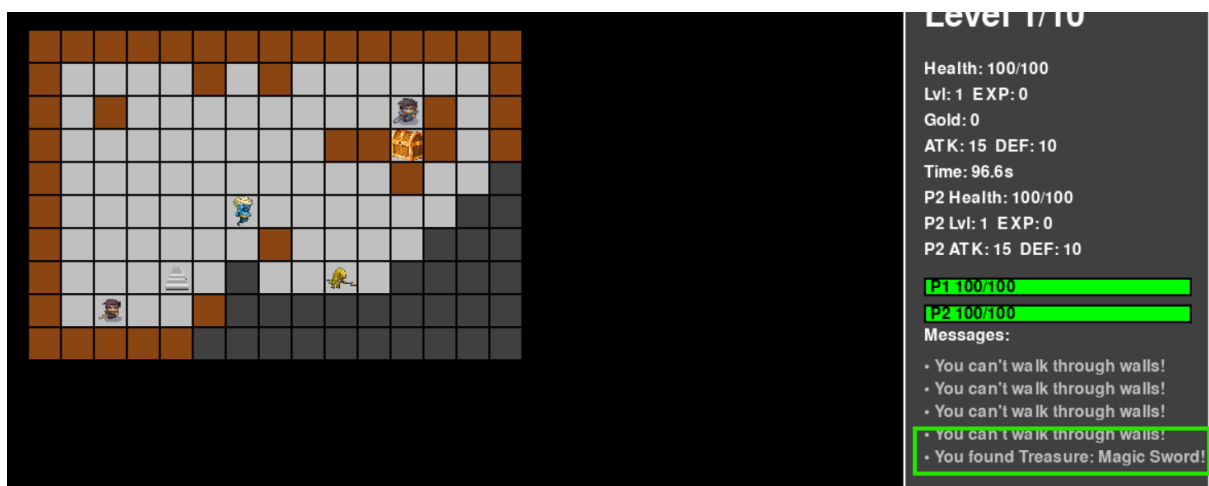


Figure 29: Obtaining treasure in GUI

Treasure chests appear as interactive graphical objects. Collecting them provides rewards such as items, gold, or quest-related progress, reinforcing exploration.

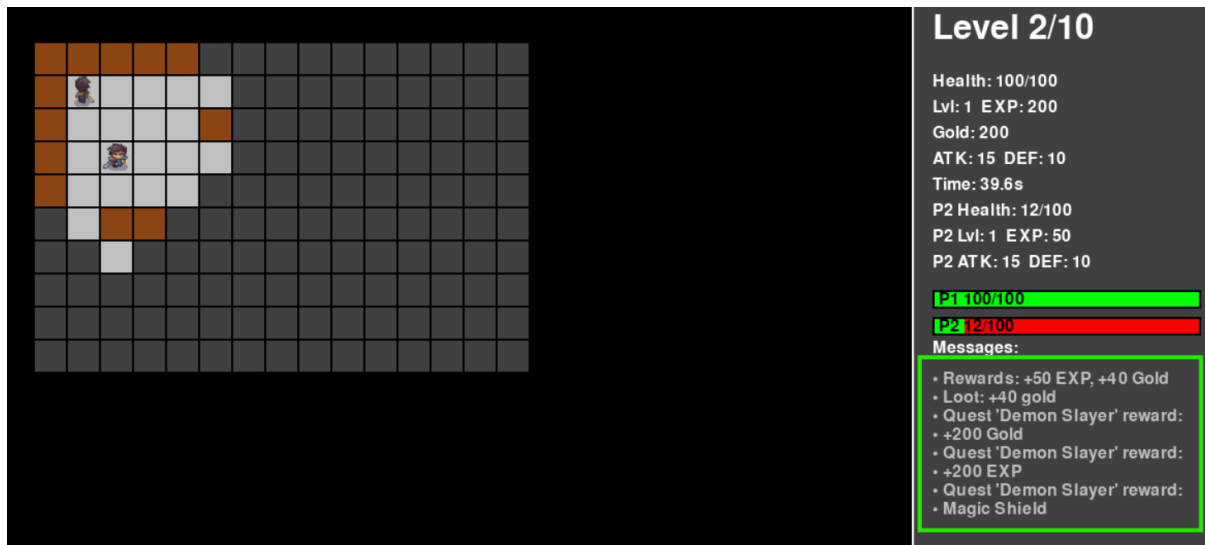


Figure 30: Quest completion mechanism in GUI

When a quest is completed, the GUI updates the quest log visually and provides rewards. This graphical feedback increases immersion and satisfaction.



Figure 31: Quests in GUI

The quest menu provides a graphical list of active and completed quests. Players can track objectives more easily compared to the text-only CLI system.

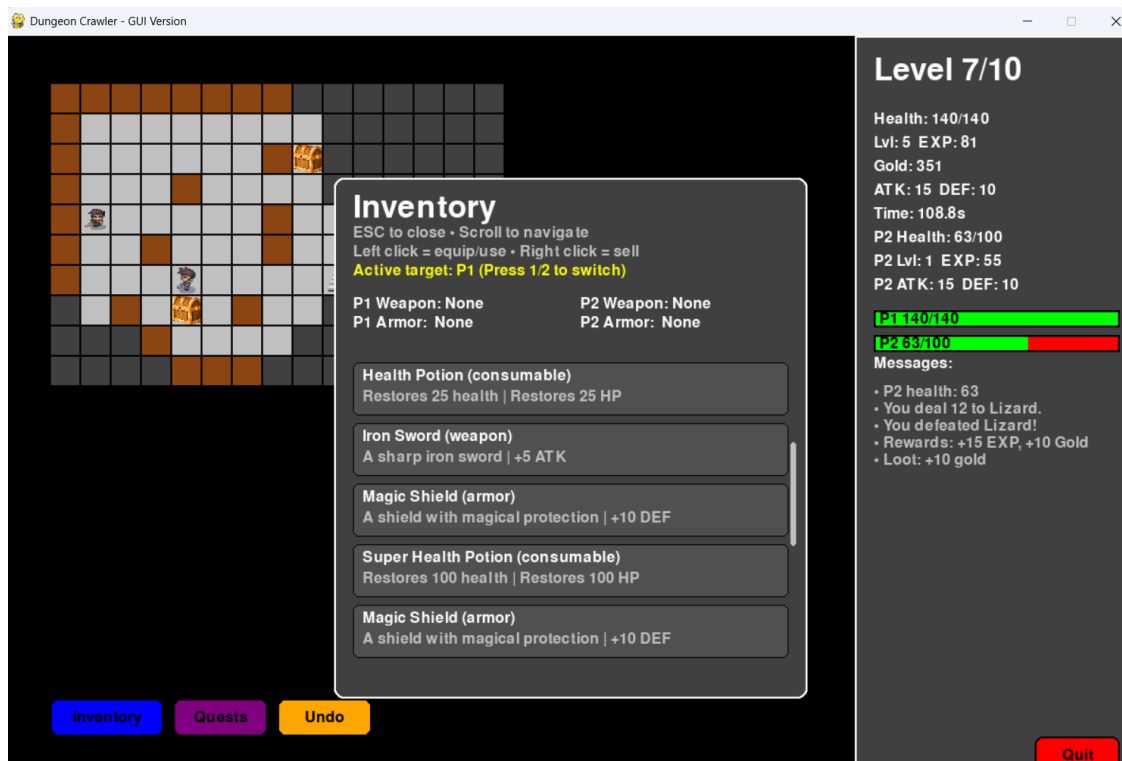


Figure 32: GUI inventory

The inventory screen displays player-held items as well as the equipped items of both the player 1 and player 2 if multiplayer is on.

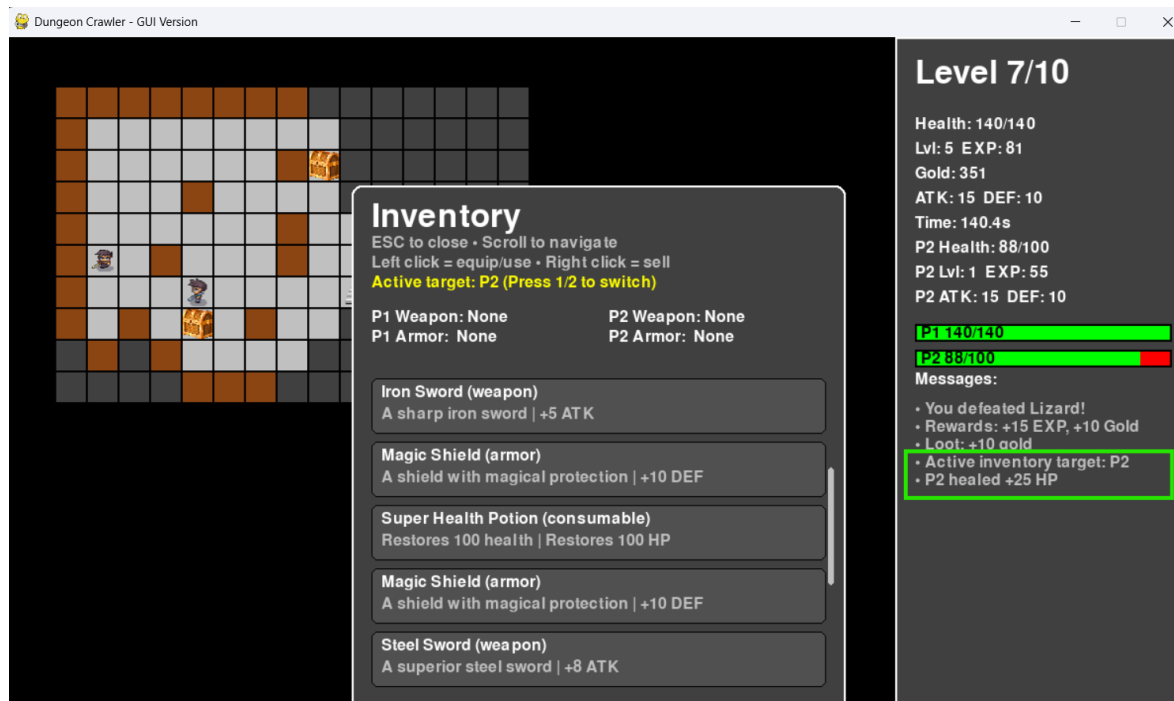


Figure 33: GUI healing mechanism.

Healing is visually represented through animations or bars. Using potions or items restores the player's health in an intuitive, easy-to-read manner.

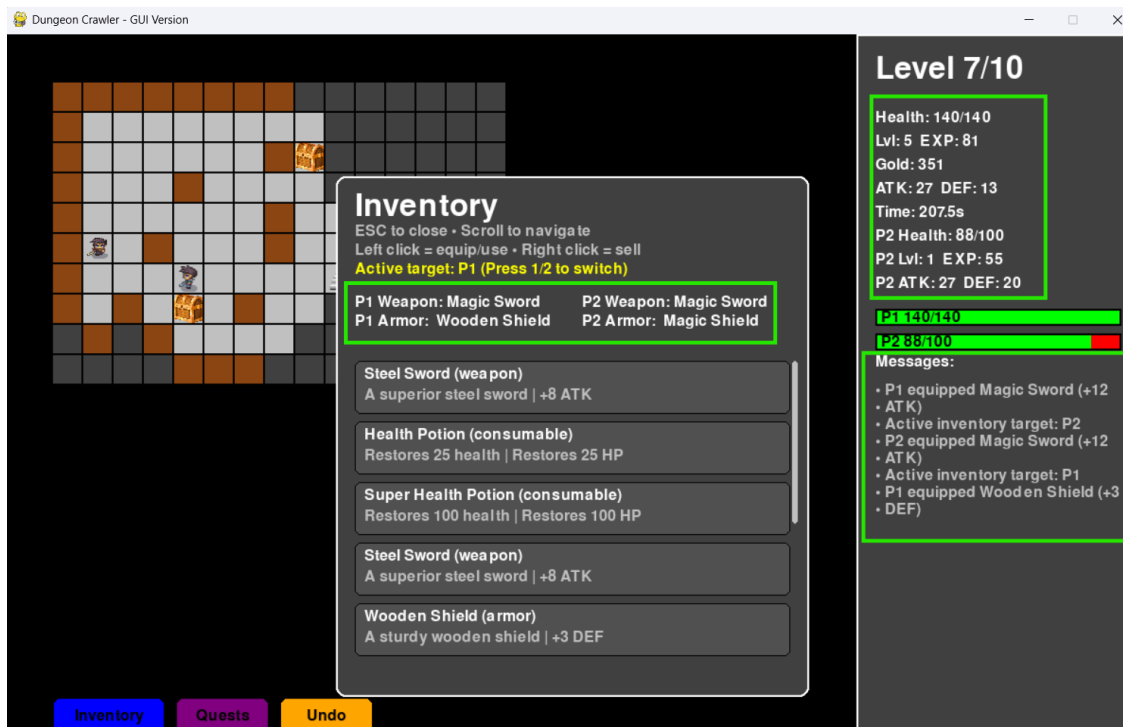


Figure 34: Equipping items in GUI

Equipped items directly update player stats in real-time on the GUI.



Figure 35: Selling items

The GUI includes an option to sell items for gold. This adds an economy system where players can trade unused equipment for valuable resources.

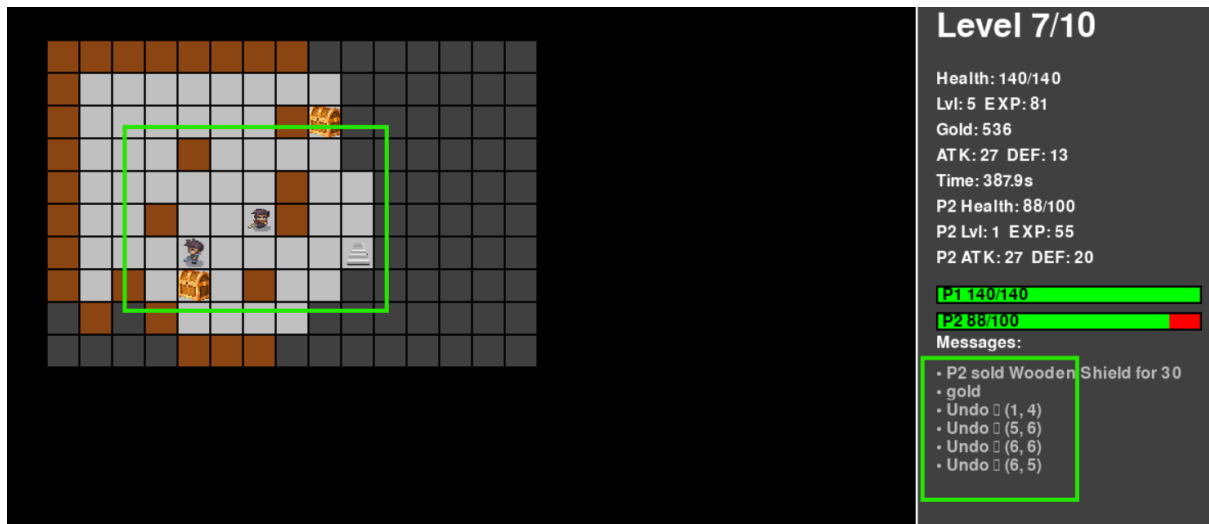


Figure 36: GUI undo button in action

An undo button provides the same functionality as the CLI's undo feature. It allows players to reverse their last move, now implemented through a clickable graphical interface.

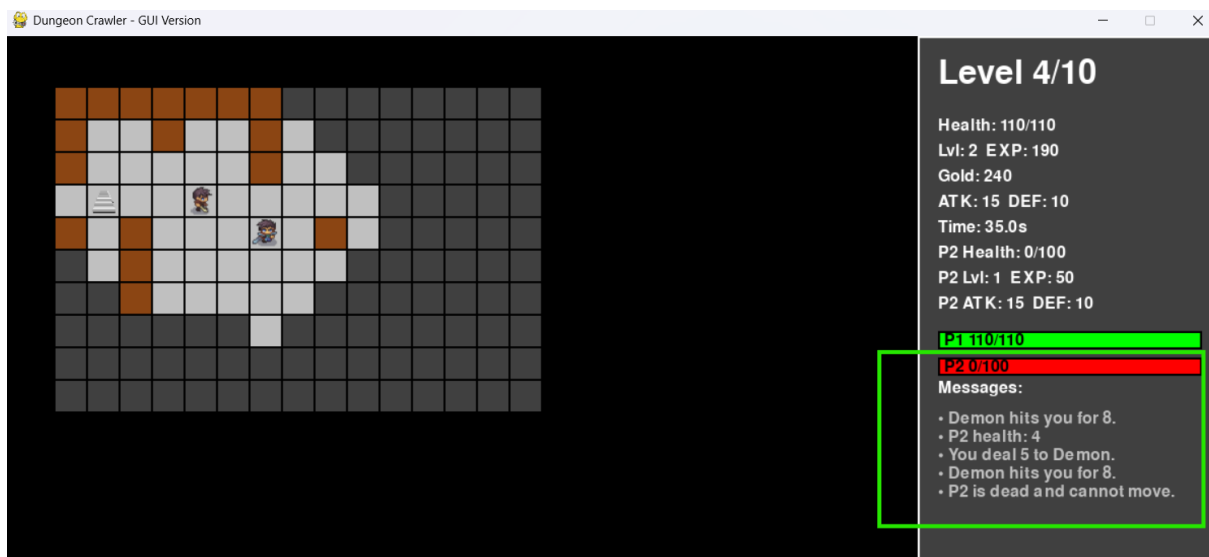


Figure 37: When a player dies in multiplayer

In multiplayer mode, if one player dies, it directly impacts cooperative gameplay. The GUI shows a visual death indicator, informing the team of the loss as the dead player cannot move or equip items.

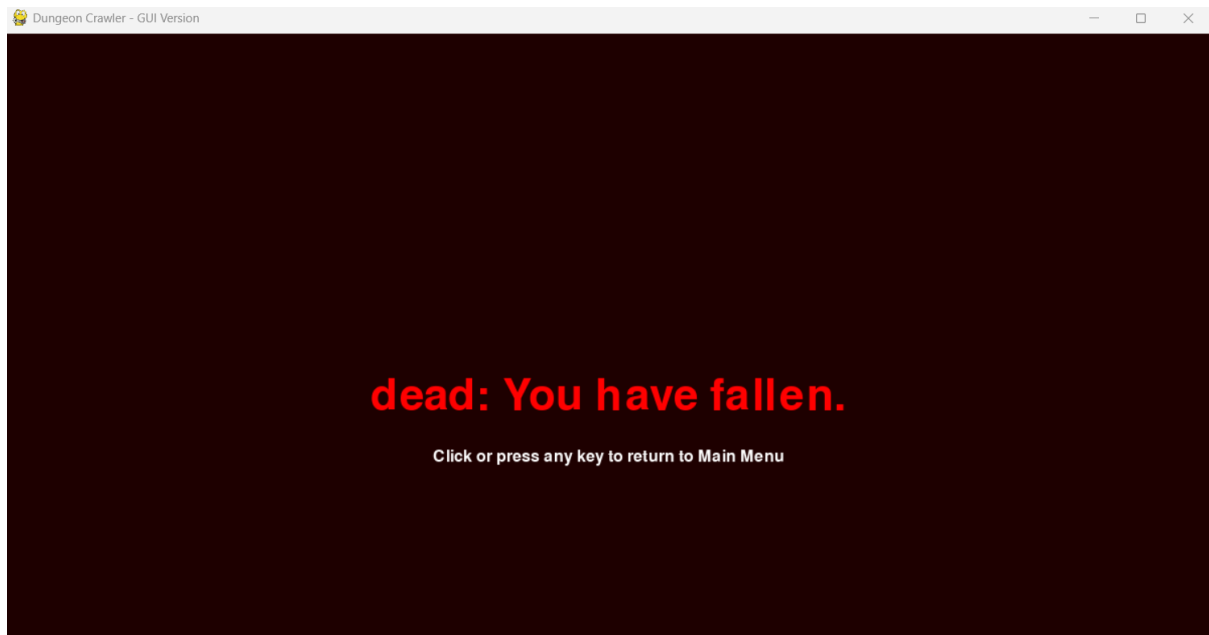


Figure 38: Death Screen

When all lives are lost, the game transitions to a death screen. This provides closure to the current run, signaling game over in a dramatic and immersive way.

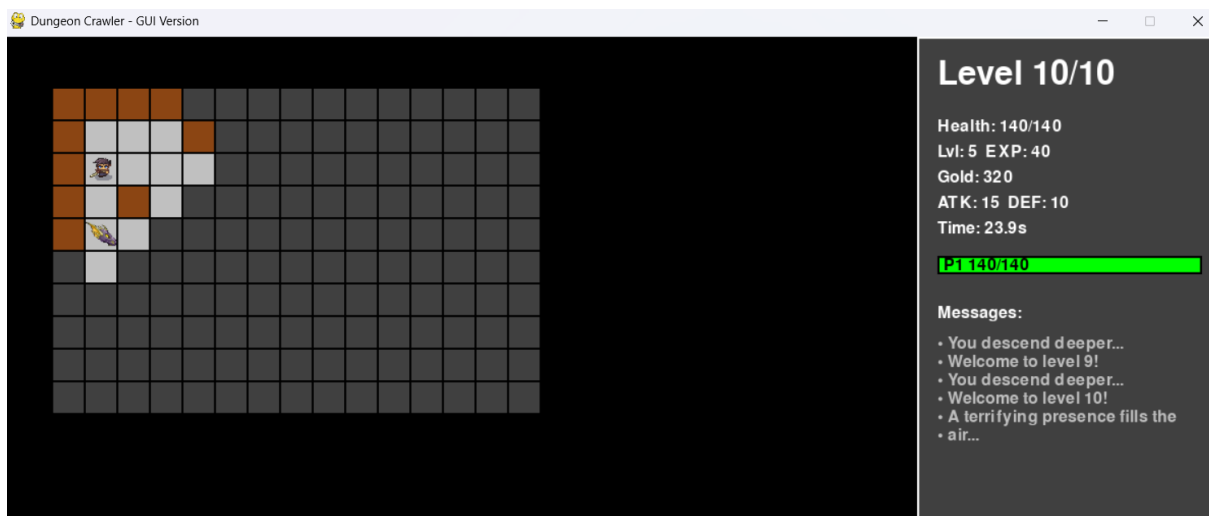


Figure 39: Final dungeon level

The last dungeon floor is represented as the most challenging area. It contains the final boss, which acts as the ultimate test of the player's skills and preparation.

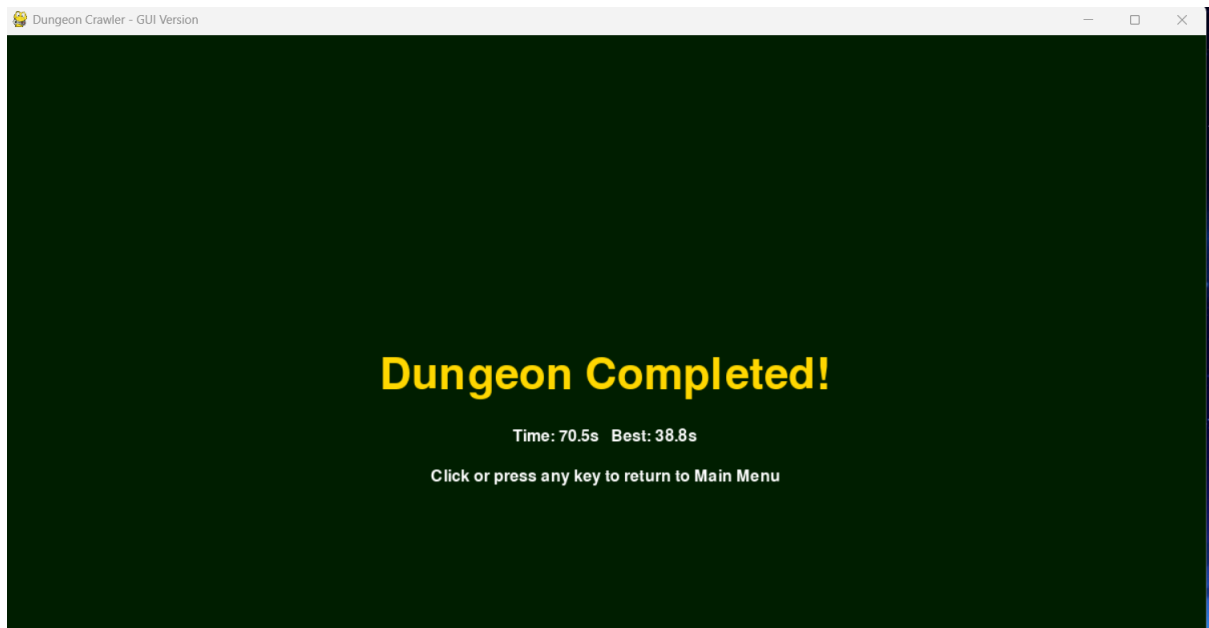


Figure 40: Victory screen

Upon defeating the final boss, players are rewarded with a victory screen. This screen acknowledges the accomplishment and signifies the successful completion of the game.

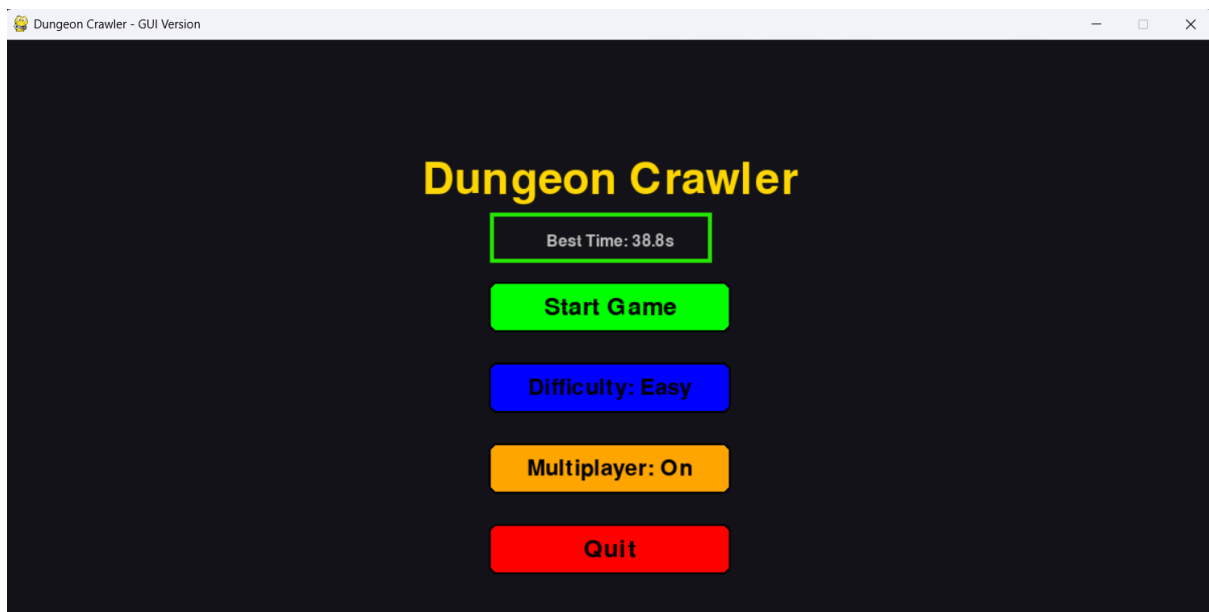


Figure 41: High score

The high score feature tracks player achievements across multiple runs. It encourages replayability and competition by recording performance metrics.

9. Conclusion

Development of the Dungeon Crawler project has been theoretical as well as a hands-on exercise of core programming concepts to a single software program. Starting from simple data structures such as lists and matrices, the project evolved to a full-fledged text-based adventure game with additional features of a fog of war system, inventory, quest logging, turn-based combat, and undo/backtrack. Data structures like recursion, linked lists, stack, and queue that were being directly implemented in game mechanics. This approach not just enforced theoretical understanding but also demonstrated how theoretical ideas could be implemented in working models.

Having both a graphical and text interface revealed how the same fundamental mechanics existed in alternative forms for different needs of users, ranging from simplicity and retro charm in the CLI to convenience and usability in the GUI.

Pedagogically, the project was insightful in terms of balancing problem-solving with creativity. The addition of features such as monster pathfinding, turn-based action queues, and undo stacks showed how theoretical principles could be put into practice for actual game design problems. At the same time, problems encountered such as debugging indexing bugs, handling multiple command mappings, and keeping the CLI and GUI consistent presented challenges that required testing, incremental development, and adaptability in programming.

Overall, the Dungeon Crawler project achieved its desired goals and provided a great basis for extension. Some possible additions are a more advanced leveling system, conversation with non-player characters, smarter monsters with AI, and saving and loading that carries over. It has also heightened not only technical competence but also problem-solving skills, flexibility, and design-thinking abilities and all skills that will be required of future software development projects.